

Data Modelling

User interfaces

UI appl helps us connect with the backend server.

2 kinds of UI appl available in MDG to perform the operations of Master Data.

1. NWBC (NetWeaver Business Client)

- a. UI appl are developed using web Dynpro, ABAP, FPM.

2. SAP Fiori

- a. Two types of appl:
 - i. Same as NWBC - called as NWBC/WebDynpro integrated appl – developed in web Dynpro, ABAP, FPM and integrated to Fiori launchpad – contains all the key info or UI components to create any master data.
 - ii. Native Fiori Appl – called as Lean Appl – completely developed using UI5 & Fiori - contains only the key information to create any master data.

Environments

SAP ECC (MDG software components are not pre-installed).

S/4 HANA (MDG software components are pre-installed [in-built] but you need to configure it first).

Storage area

Example: hard disk or a container.

Everything in SAP will be stored in SAP database table.

Purpose: to save the data into the relevant underlying database tables.

In MDG, other than standard table (KNA1, LFA1, CSKS, MARA, etc), we'll have other tables as well.

Example: /1MD/MD____001 ? MM check table material. System generated table.

USMD_ACTIVE field in the above table ? 0 (Active, active data) or 1 (Inactive, being processed in a CR).

When CR is being processed, the data is stored into these generated table in the form of inactive data.

These MDG generated tables are called **Staging Areas/Tables**.

- Active data.
- Inactive data.

Core master data table (KNA1, LFA1, etc) = **Active area** tables.

- Basis data.
- Organization data.
- Master data.
- Transactional data.

Deployment Option

A deployment option tells you how we are installing and implementing MDG in SAP environment of the client.

Different deployment options in MDG:

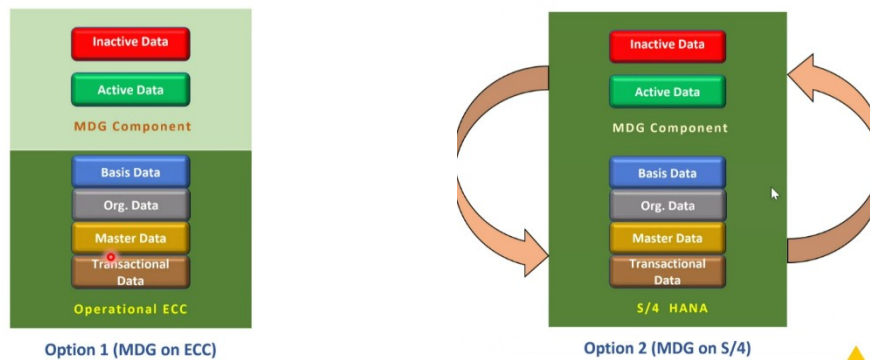
- Hub

- Co-deployment
- Cloud deployment (latest; not widely used yet)

Co-deployment (local MDG):

Deployment is based on i. central MDG, local MDG and ii. storage area.

Local MDG is on top existing operational system or legacy system MDG is installed and implemented. Central MDG on operational/legacy/transactional systems are untouched. MDG is a separate system. MDG software components are installed & implemented in the ECC or S/4 HANA system and we'll integrate all the legacy systems with MDG. MDG is centrally governing the master data of all the systems.



Environment 1 (MDG on ECC):

MDG can be installed in ECC but the min version of ECC is 6.0.

On top of operational ECC, MDG software components can be installed.

Data is exchanged between MDG component data and Operational ECC data.

MDG is present on top of operational ECC system. Present in the same system, not a separate system.

When creating material, basic details like material description, group, plant etc need to be entered.

These details MDG can get from the underlying operational ECC.

Once the CR is approved and the data is active, we need to send this approved data to respective business systems for their business transactions.

Environment 2 (MDG on S/4HANA):

In S/4HANA, MDG is pre-installed.

Hub Deployment (Central MDG)

MDG Hub on ECC:

We have an operational system (sap or non-sap) and a MDG system having basis data, org data and master data. Once data is governed in MDG, data is replicated back to the operational system.

Example: All the material data exists in ECC. But if you want to create a material or change a material belonging to the same plant or same material type, these plant or material type should be present in the MDG system. Meaning, reference data should be in sync with the MDG system (reference data sync-up). Replicating the data back to the operational system after the governance is done. This replication can be done via idocs/rfc/soa/file. Known as communication channels.

MDG hub on S/4:

No difference except software component.

Hub Deployment Option

Co-deployment Option

When needed? Whenever there is a central governance required when there are multiple legacy/business/operational/transactional systems.	Whenever there is a limited number of systems, and all are SAP systems.
Integration with multiple SAP/non-SAP systems.	Integration only on SAP systems.
Independent of transactional data. No effect on the transactional data. Replication of transactional data after it is active to the operational system can be stopped if needed.	Affects transactional data immediately. Replication happens immediately cannot be stopped if the data is wrong.
Easy maintenance in case of multiple legacy systems. Example: you want to implement a new enhancement, no need to implement this same enhancement in multiple systems.	Not easy to maintain as it must be implemented in each of the legacy systems. Example: you want to implement a new enhancement; you need to implement this same enhancement in multiple systems.
Easy to upgrade since only 1 MDG system.	Tedious to upgrade as there might be multiple MDG systems. If something goes wrong, it will affect the transactional data.
Initial data load (bringing the data from operational system to MDG system) is required.	Initial data load is not required.
Replication of active data back to the legacy system is required.	Replication not required. But for finance data replication is needed. It's called as self-replication.
Implementation cost: in case of single transactional system, the implementation cost is high compared to local MDG. in case of multiple transactional systems, the implementation cost is less compared to local MDG.	Implementation cost: in case of single transactional system, the implementation cost is less compared to central MDG. in case of multiple transactional systems, the implementation cost is high compared to central MDG.
No. of resources: in case of single transactional system, the no. of resources is same as local MDG. in case of multiple transactional systems, the no. of resources is always less compared to local MDG.	No. of resources: in case of single transactional system, the no. of resources is same as central MDG. in case of multiple transactional systems, the no. of resources is always more compared to central MDG.

Data Flow between staging area and active area

Staging area and active area behavior for MM, BP is different when compared to FI.

Creation:

Behavior for MM:

Staging table ? /1MD/MD____001

Active area table ? MARA

When Requester creates a CR for material, the material number is prefixed with \$ sign and it is stored in the staging table with USMD_ACTIVE field value as 0 (inactive). Once the CR is approved (Final Check Approved), the material number is generated. The entry from the staging table is removed and the actual material number is updated in MARA table (active area table).

Behavior for FI:

Staging table for Company ? /1MD/MD____06M

Active area table for Company ? T880

When CR is created for Company, unlike for material object no temporary number assignment is done for FI objects. After the CR is approved, the data remains in staging table with USMD_ACTIVE field value as 1 (active) and data is not yet updated in the active area table.

Change:

Behavior for MM/BP:

Sources of search by the Framework: Staging area and active area.

Once the change request is initiated, staging area gets updated with two records:

USMD_ACTIVE 0: inactive data which is in governance process.

USMD_ACTIVE 1: active data (before changing CR).

This is because, if the CR is rejected then the framework has to rollback all the changes to the original version (active version). Once the CR is approved, the data from staging area is removed and active table data is updated.

Behavior for FI:

Sources of search by the Framework: Staging area.

Staging area gets updated with two records:

USMD_ACTIVE 0: inactive data which is in governance process.

USMD_ACTIVE 1: active data (before changing CR).

Once the CR is approved, the inactive data is removed and only the active data remains (changed data).

MDG Architecture Principle

Two architecture principles:

Re-use active area/mode (non-finance) – existing structure of applications are used. For example, MDG for material makes use of the MARA table in ECC.

Generated active area / Flex mode (finance) – tables as defined in the MDG data model (staging tables) are used to store active data.

Data Modelling

Data segments for MM:

Material header data (entity type; can be correlated to table in ECC or S/4H) ? Main data segment ? MATNR, MTART, MATKL, BISMT etc (attributes).

Material plant data (entity type) ? dependent data segment ? MATNR, WERKS, Plant specific status etc (attributes).

Material Plant's storage location data (entity type) ? dependent data segment ? MATNR, WERKS, LGORT, LVORM etc (attributes).

Classification & characteristics data (entity type) ? Class, characteristics, etc (attributes).

Staging tables are not manually created but auto generated based on the design of the above data segments in the system.

Data Model Design:

What are the entity types, in each data segment what will be their attributes, what is the dependency levels, hierarchy levels, and what is their triggering point.

Data model design ? build the data model ? build UI ? users can access the application to govern the materials.

Custom Data Model Creation Steps:

1. Data model entry creation.
2. Determination of flex or reuse mode.
3. Create custom data model in the required mode.
4. Create entity types.
5. Add attributes to the entity types.
6. Define the relationships.
7. Create business object type.
8. Fill namespace/prefix for structure generation.
9. Fill package for structure generation.
10. Assign business object to the data model.
11. Generate staging area (staging tables).
12. Generate staging structures.

Types of Entity Types:

- Storage / Use (SU) Type 1 Entity – changeable via change request; generated database tables.
- Storage / Use (SU) Type 2 Entity – changeable w/o change request; generated check/text tables.
- Storage / Use (SU) Type 3 Entity – not changeable via MDG; no generated tables.
- Storage / Use (SU) Type 4 Entity – changeable via other entity type; generated database tables.

SU Types Storage

Use

Type 1	Generated db tables (all below staging tables are generated)	Changeable via CR (only this should be linked to CR)
Type 2	Generated check/text tables	Changeable w/o CR (no CR process)
Type 3	No generated tables	Not changeable via MDG
Type 4	Generated db tables (not all below staging tables are generated)	Changeable via other entity type (should be associated to type 1 entity directly or indirectly).

Staging tables are generated tables by the framework for an entity of a data model.

Types of staging tables:

- Check table
- Mapping table
- Text table
- Table with further attributes
- Attachments

Types of Relationships (between the entity types)

- Leading relationship
- Qualifying relationship
- Referencing relationship
- Foreign relationship

Types of Attributes

- Key attributes
- Non-key attributes

MARA
MATNR
LVORM
MTART
MBRSH

MARC
MATNR
WERKS
PSTAT
LVORM
BWTTY
MAABC
EKGRP

MATKL
BISMT
WRKST
MARD
MATNR
WERKS
LGORT
PSTAT
LFMON
LABST
UMLME

Root table is always Type 1 entity.

Here, MARA is the type 1 entity.

Once material is present, we can create/extend materials to plant. Then, extend plants to storage location. Hence MARC and MARD are called as dependent tables.

Entities which cannot be present w/o Type 1 entities are called as Type 4 entity. MARC and MARD are Type 4 entities. Dependent on Type 1 entity.

Here, MARA is the parent of MARC and MARC is the parent of MARD.

Hence, MARA leading MARC and MARC leading MARD.

Key attributes of parent entity will become key attributes of child entity.

How to define key field for Type 1 entities:

Can be defined directly by checking the key field check box.

For Type 4:

not allowed. Key attributes cannot be defined directly for type 4 entities. If you want to create a key field for type 4 entities, do it via relationships.

For Type 3:

Not allowed. That's because attributes can be assigned only to type 1 and type 4 entities.

Relationship must be between two entities. Source entity is FROM-ENTITY TYPE and target entity is TO-ENTITY TYPE.

Cardinalities – no. of instances of FROM and TO entities in a relationship.

Possible cardinalities are: - 0: N, 1: N and 1:1.

Defining additional key field for Type 4:

It can be done via qualifying relationship now.

Create type 3 entity and define qualifying relationship between type 3 and type 4.

Creating Custom Data Model:

Step 1: MDGIMG ? Data Modelling ? Edit Data Model ? New Entries ? Enter name of your data model and description ? Enter.

Data Model Properties:

1. Name
2. Description
3. Active area
4. Prefix/namespace
5. Package
6. Username
7. Changed on
8. Active version

How to know data model is reuse and flex mode?

Refer the ActiveArea column. Difference is in the flex mode the active area is itself the staging area (active data will remain in the staging area). And in the reuse mode, once the CR is activated the active data will go to the active area.

If the ActiveArea column is blank or MDG then it's a flex data model.

If ActiveArea column is other than blank or MDG then it's a reuse data model.

Always create custom active area for your custom data model.

Step 2: Select your data model and click on Reuse Active Area.

Step 3: New Entries ? enter the name of your active area and give a description ? Save.

Error "Model ZM, reuse active ZM_ACT: ABAP class cannot be used".

Long Text: The ABAP class for accessing the database tables in the reuse active area must meet the following prerequisites:

- The class must be activated.
- The class must implement the interface **IF_USMD_PP_ACCESS**.
- The instance generation of the class must be 'public'.
- The CONSTRUCTOR must only have optional IMPORTING parameters.

To resolve this, you need to create a custom class using SE24 and implement IF_USMD_PP_ACCESS interface and assign this to the active area.

Why is access class needed?

Whenever we create a data model as reuse data model, the data should be read from active area to staging area in case of read/change operations and in case of create operation, the data should update from staging area to active area. Once CR is activated, the data goes from staging area to active area via this access class.

Step 4: Enter the access class name in access class field and save.

Step 5: Enter the active area you've created just now to the ActiveArea column in Inactive Data Model. This is assignment of active area to your custom data model.

Step 6: Active Version  indicates whether data model is activated or not. All new data models will be in state “none”. The other values in this are “different” and “same”.

None means data model wasn’t activated even once.

Different means you are making changes on already activated data model. Latest active version and current version are different.

Same means activated no new changes. Latest active version and current version are same.

Creating Custom Entity Type:

To create Type 1 entity,

Step 1: Select the data model and double click on Entity Types.

Step 2: New Entries  enter the name of the entity type.

F1 technical information: Table name is V_USMD0020 and Field Name and Data Element is USMD_ENTITY. Length of the field should be of 9 characters.

Properties for the Entity Type are:

- Storage/Use Type
- Validity/Entity
- Data Element
- Is Hierarchy Type
- Validity/hierarchy
- Key assignment
- Language dependent texts
- Attachments
- Sets
- Search help
- Temporary Keys
- Active area
- Deletion

Step 3: choose the type 1 from the drop-down menu.

Step 4: Choosing Validity/Entity. Options available in the drop-down menu are Edition and No Edition.

Edition is applicable only for finance data model.

Step 5: Enter Data Element.

When you define a data element, a key attribute for this entity will be generated with the entity name itself. (Eg., for data model MM, entity type MATERIAL, data element is MATNR. So, MATNR becomes the key attribute for MATERIAL entity). Next the framework checks whether there is any temporary key assignment. If temporary key is assigned, then temporary key will be generated and stored in this key attribute.

When you enter temporary key assignment w/o giving any data element, the following error will be generated “Number range assignments are not supported”.

Internal key assignment or changeable keys are permitted only for type 1 entity with data element.

Data element for type 1 entity is mandatory if temporary keys assignment is done.

Not enough leading relationships.

Entity types with SU type 1 w/o a data element must be used as to-entity types in at least two relationships with the relationship type Leading when the cardinality 1:N is used.

Which entities can have data element assignment?

- Type 1, type 2, and type 3.

What are the scenarios in which data element assignment is mandatory for type 1 entity?

- When there is a temporary key assignment.
- If key assignment is changeable (key can be changed; no internal key assignment).

Step 6: Select the Key Assignment as Key can be Changed; internal key assignment possible.

Consider the below scenario:

Material creation 3 step process flow.

Change Request – 1000

	Key cannot be changed; no internal key assignment	I - Internal key assignment only	C - Key can be changed; no internal key assignment	B - Key can be changed; internal key assignment possible
Requester	Key is external i.e., entered manually.	When CR is created, material number is pre-fixed with dollar sign. Eg., \$3456. Done using the path MDGIMG ? Data Modelling ? Define prefixes for Internal Key Assignment	Key is external i.e., entered manually. Eg., 1234	Internal key is generated e.g., \$123
Reviewer	Key remains same.	Key remains same i.e., \$3456	This key can be changed e.g., 1235	Key can be changed e.g., 124
Approver	Key remains same.	Key remains same i.e., \$3456	This key can be changed again e.g., 1236	Key can be changed again e.g., 125
Activation	Key remains same.	Key is replaced with the actual next available material number.	Last key is used as the material number i.e., 1236	Last key is used as the material number i.e., 125

Step 7: Language dependent texts.

Text table will be generated only if you select this option. If this is selected, the system includes the language as a key field when it generates the store for entity texts. Can be used only for type 1 or type 2. Can be used when you need to have description in different languages.

Step 8: Attachments.

If this indicator is selected you can store attachments (e.g., Microsoft Word or Adobe PDF files) to entities that belong to this entity type. The system automatically provides a data store for this. Can be selected only for Type 1. The attachments are stored in a separate database table.

Step 9: Search Help.

Step 10. Source Field.

Specified only for type 3.

T-code SNRO to create a number range for temporary key.

Step 11: Deletion:

Deletion of entities not allowed.

You cannot delete instances of entity types that have internal number assignments or changeable keys.

Not allowed for type 1 entities.

Creation of Type 4 entity:

Creating ZMARC as type 4:

just select type 4 and give a description. Other properties are not important for type 4.

Relationship of type "leading" inconsistent.

Define a "qualifying" relationship for entity type ZMARC.

Add attributes to this entity type. You cannot define the key fields directly for type 4.

While saving, you will get the below errors while saving.

Data element of (hierarchy) attribute LVORM is not unique.

Attributes or hierarchy attributes with the same name must reference the same data element for all entity types. It means we have the same attribute LVORM in different entities in the same data model, but the data element is different. So, check the data element and change it accordingly.

Extra note: LVORM means deletion flag. It is present in both MARA table with data element LVOMA (description: deletion flag at client level) and in MARC table with data element LVOWK (description: deletion flag at plant level). As mentioned above, if you choose to use LVORM in both of your custom data model ZMARA and ZMARC then the data element must be same. If you want to use different data element, then you have to change the name of the attribute.

For example:

Data model MM

Entity type MATERIAL ? Attribute LVORM_MAT ? data element LVOMA.

Entity type MARCBASIC ? Attribute MARCLVORM ? data element LVOWK.

Entity type ZMARC, attribute PRCTR: Invalid check table.

There is an attribute with a data element assigned that refers to a check table. Besides the client, the check table can only have one key field that corresponds to the attribute.

To see the check table of PRCTR, open this field in SE11. Go to Value Range tab. Below you can see Value Table CEPC. This is the check table for PRCTR. Double click on CEPC and you can see 3 key fields are there including PRCTR as one of the key fields. To resolve this, for the key fields create type 3 entity ZDATBI, ZKOKRS and ZPRCTR and maintain relationships for these.

ZPRCTR referencing ZMARC 0: N. (referencing is used as PRCTR is non key attribute of ZMARC).

Once you save the above changes, you will get another error “**data model ZM entity type ZPRCTR: Number of leading entity types is invalid**”. For the entity types with SU type 3 for which the check table is predefined by the data element or domain, the other key fields in the check table must be mapped using relationships with Leading as the relationship type. To resolve this,

ZDATBI leading ZPRCTR 1: N.

ZKOKRS leading ZPRCTR 1: N.

Once you save this, you will get the error message “**entity type ZMARC, rel PRCTRMARC: Further relationship to entity type ZDATBI missing**” and “**entity type ZMARC, rel PRCTRMARC: Further relationship to entity type ZKOKRS missing**”. To resolve this, create below two relationships.

ZDATBI referencing ZMARC 0: N.

ZKOKRS referencing ZMARC 0: N.

Relationship of type “leading” inconsistent.

Entity type 4 must be used as to-entity types in one relationship with the relationship type Leading. You can use the from-entity type to process the attributes or assignment of additional entity types using the relationship type Referencing. There is either more than one relationship with Leading as the relationship type, or none at all. Make sure that there is exactly one relationship with the relationship type Leading. ZMARA LEADING ZMARC 1: N. After maintaining the relationship, you might get below new errors.

Entity type ZMARA: Attribute LVORM is assigned more than once.

Entity type ZMARA: Attribute PSTAT is assigned more than once.

Within a single hierarchy, you cannot have multiple attributes with the same name. LVORM and PSTAT are there in both ZMARA and ZMARC. Because of this, we are getting these errors.

To resolve this, create type 3 entities ZLVORM and ZPSTAT and maintain referencing relationships. After creating, removing the previous attributes from the entities ZMARA and ZMARC.

ZLVORM referencing ZMARA 0: N

ZLVORM referencing ZMARC 0: N

ZPSTAT referencing ZMARA 0: N

ZPSTAT referencing ZMARC 0: N

Define a “qualifying” relationship for entity type ZMARC.

While maintaining the relationship ZMARA leading ZMARC 1: N, if you change the cardinality to 1:1 then you won’t get this error. When you choose 1: N, then to identify all N materials uniquely then you need to have additional key fields other than MATNR. Additional key fields not required for 1:1.

To resolve this, create additional key field for type 4 entity by creating a type 3 entity ZWERKS (data element is mandatory) and defining the relationship. ZWERKS qualifying ZMARC 1: N (cardinality for this should be the same as leading).

While creating ZMARD entity type 4, you will get the below additional errors:

Entity type ZMARD, attr. LABST; Attribute missing for currency/unit.

Data type of LABST (stock) is QUAN (quantity). Whenever you create fields using QUAN data type, you need to link this quantity field to the unit field (as quantity should refer to the unit also).

To do so, define unit field in any of the entities of Data Model’s Type 1 hierarchy. Then go to your quantity attributes. You can see a column “Curr./UoM Field”. Give the unit field name there and save it.

Entity type ZMARD, attr. RETME; Attribute missing for currency/unit.

Entity type ZMARD, attr. SPEME; Attribute missing for currency/unit.

Entity type ZMARD, attr. UMLME; Attribute missing for currency/unit.

Business Object Type:

When you make any changes to the attribute (for example, changing the description of the attribute), you will get the below warning messages. These warning messages should not be ignored.

Data model ZM; fill namespace/prefix for automatic structure generation.

Data model ZM; fill package for automatic structure generation.

To resolve these, you need to fill namespace/prefix and package for structure generation.

Business Object Type Code is a central component which connects all the objects of a MDG solution. It's an alpha numeric value which determines whether a data model, UI data model etc. is for BP or MM or other some object.

Path: MDGIMG > General Settings > Data Modelling > Define Business Object Type Codes.

BO Types can be assigned to Type 1 entity only.

Define prefix for internal number assignment:

Prefixes are to be defined only if internal number/temporary keys assignment is there for type 1 entity.

This is done so that the framework knows which one is the temp key and which one is the final key.

Path: MDGIMG > General Settings > Data Modelling > Define prefix for internal number assignment. if you use the setting Internal Key Assignment Only but do not define a prefix, the system uses the prefix \$ by default. This is not mandatory as by default the prefix will be \$.

Generating staging area and staging structure:

Staging table/staging structure in MDG is not created manually. Instead, framework generates the staging tables (staging area) & staging structures.

When/how the staging tables would be generated?

After the activation of Data Model.

When/how the staging structures would be generated?

Case 1: manual staging structure generation

Select your data model and enter the prefix/namespace and package.

Case 2: auto staging structure generation > activate the data model + define prefix/namespace and package for structures. Select the data model and click on "Prefix and Packages for Model Enhancements. This is done only when you make some enhancements to the standard data model.

USMD_DATA_MODEL > execute this report to see the generated staging tables for each data model.

Entity Type column will always have type 1 and type 2 entities.

Included Entity Type column will have the type 4 entity (dependent on type 1 entities).

Subtype column:

Check table – for type 1 entities.

Text table – will be generated if the type 1 entity is language dependent.

Table with Further Attributes – always generated for type 4.

Mapping table – for type 1 entities.

Logical Name column > template: <type><subtype>_<data_model>_<entity_name>. Example:

TCK_ZM_ZMARA, TXT_ZM_ZMARA, TOA_ZM_ZMARA_ZMARC, TMX_ZM_ZMARA.

Physical Name column > These staging tables are not transportable (meaning saved in local package; not present in QA or prod systems). Hence, we need to generate these staging tables in every environment i.e., we need to activate the data model manually in every environment (cut over activity). These are transparent tables in the system.

How is check table generated?

Structure of the staging table ? what are the fields available in the staging table.

Double click on any of the staging table. The fields are the attributes maintained for that data model.

The fields are prefixed with a specific format.

/1MD/<data-model-name><attribute>.

Additional fields which are not in the data model but present in the staging tables are:

USMDK<data-model><entity-type> ? data element USMD_Tech_Key ? **Technical key** ? key field used to uniquely identify a record.

Use case example:

ZMARA type 1 entity

ZMARC type 4 entity

ZMARD type 4 entity.

ZMARA leading ZMARC and ZMARC leading ZMARD.

ZMARA has key fields ? key1 key2 key3

ZMARC has key fields ? key fields of ZMARA (key1 key2 key3) + key4

ZMARD has key fields ? key fields of ZMARA (key1 key2 key3) + key4 + key5

Instead of maintaining the key fields in all the dependent entities, maintain 1 key field for all the key fields. Maintain 1 key field for ZMARA entity. That key field is known as technical key. The value is generated randomly by the framework.

key1+key2+key3 = technical key

This is done only for the type 1 entity.

The screenshot displays the SAP Dictionary: Display Table interface for the table USMDKMMATERIAL. The table structure is shown in the main window, and the 'Data Element Attributes' dialog is open on the right.

Field	Key	In...	Data element	Data Type	Length	Dec...	Coordinate	Short Description
MANDT	✓	✓	MANDT	CLNT	3	0		Client
USMDKMMATERIAL	✓	✓	USMD TECH KEY	INT4	10	0		Technical Key
/1MD/MMWERKS	✓	✓	WERKS_D	CHAR	4	0		Plant
USMD_ACTIVE	✓	✓	USMD_ACTIVE	CHAR	1	0		Active/Inactive Data
/1MD/MMFEVMAWCR			FEVOR	CHAR	3	0		Production Supervis
/1MD/MMSFMAWCR			CO_PRODPER	CHAR	6	0		Production Scheduling
USMD_O_MARCWRKSD			USMD_OBSELETE	CHAR	1	0		Obsolete Data Record
/1MD/MMBASMG			BASMG	QUAN	13	3		Base quantity
/1MD/MMBEARE			BEARE	DEC	5	2		Processing time
/1MD/MMDEIT			DEIT	DEC	3	0		In-house production
/1MD/MMFRTIME			FRTIME	UNIT	3	0		Production unit
/1MD/MMKZPRO			KZPRO	CHAR	1	0		Indicator: withdraw
/1MD/MMMATGR			MATNRGROUP	CHAR	20	0		Group of Materials f
/1MD/MMOCMPF			OCM_PROFILE	CHAR	6	0		Profile for OCM PP /
/1MD/MMRUEZT			RUEZT	DEC	5	2		Setup and teardown

The 'Data Element Attributes' dialog for 'USMD_Tech_Key' shows the following details:

- Data element: USMD_Tech_Key
- Short Description: Technical Key
- Domain: INT4
- Data Type: INT4
- No. of Characters: 10
- Sign: ☒ Sign
- Internal Format: ABAP type 1, Internal Length 4
- Output Length: 11
- Default field name:
- Input Help: P Input help implemented with check table
- Search Help Name:
- Fixed val. ex: ☐ Fixed val. ex
- Foreign key exists: ☒ Foreign key exists
- Check table: /1MD/MM...0F9
- Database Attributes: ☒ Key field, ☒ Initialize

How is text table generated?

In the text table, short, medium and long text descriptions are generated.

Here, language key, technical key and active/inactive data record are the key fields.

How “table with further attributes” is generated?

Here technical key, USMD_ACTIVE and additional key(s) field of type 4 are the key fields. These tables are dependent on type 1 entities.

How “mapping table” is generated?

Mapping of staging key (temporary key of material before CR is finalized) with the actual material number (after CR is finalized) is stored in this table.

1/MD/MMMATERIAL → final material number

USMDTMMATERIAL → temporary key.

What will happen to the staging table data during system refresh?

Staging table name is different across the landscape but the structure is same.

Production refresh → everything will be loaded from the scratch. Entire system will be cleaned and then re-storing the backed-up data.

Options:

- This production data can be backed up in a clone production system and once refresh is done, load this data back to production system.
- Or you can create a dummy client and back up the production data in this client. Once refresh is done, load this data back to the main client.

Staging structures:

Prefix and package assignment while creating data model is mandatory for generating staging structures.

Staging structures are generated in 2 ways:

- automatic generation (recommended) □ when prefix and package are assigned at data model level.
- Manual generation (not recommended) □ when no prefix and package are assigned at data model level. MDGIMG → General Settings → Data modelling → Generate data modelling specific structures. Enter the structures manually there.

Select the data model and double click on “Data Model Specific Structures”. These are the structures generated by the framework for that data model.

Where-used column → type of the structure.

Types of structure:

1. PDF based Forms

- a. PDF_<entity-name>
- b. SAP has a feature of allowing users to download the entire data present in the CR.
- c. Relevant for type 1 and 4.

2. SMT mapping for replication

- a. <entity-name>
- b. Used for replication.
- c. Relevant for type 1 and 3.

3. Mapping of reuse active area

- a. PP_<entity-name> (PP stands for primary persistence)

- b. Used for both type 1 and type 4.
 - c. Used very frequently.
 - d. Reflects exactly your entity as such.
 - e. Sometimes we write logic to read entity data or update entity data. In the code logic, the internal tables used will be declared using this structure.
- 4. Data replication framework**
 - a. DRF_<entity-name>
 - b. Contains the keys from the main entity type.
 - c. Relevant for type 1 and 3.
- 5. Search application**
 - a. ES_<entity-name>
 - b. For the search results.
 - c. Relevant for type 1 and 4.
- 6. Field control for attributes**
 - a. FC_<entity-name>
 - b. The data type (component type) for all the components (attributes) is USMD_UICTRL_PROP except for MATERIAL. For MATERIAL it is MATNR.
 - c. Relevant for type 1 and 4.
- 7. Field properties for attributes and key fields**
 - a. FP_<entity-name>
 - b. The data type (component type) for all the components (attributes) including MATERIAL is USMD_UICTRL_PROP.
 - c. Relevant for type 1 and 4.
- 8. Key fields on an entity type**
 - a. KF_<entity-name>
 - b. Only fields of that entity type.
 - c. Relevant for type 1 and 4.
- 9. Hierarchy attributes**
- 10. Data extraction**
 - a. EX_<entity-name>
 - b. Used to export the data from MDG system to files.
 - c. Relevant for type 1 and 4.

Type 1 entity will have all the above structures except 9 hierarchy attributes.

Data Type USMD_UICTRL_PROP:

Can control field properties in the UI dynamically (at runtime).

It is of character string type of length 1.

Dictionary: Display Domain

Domain: USMD_UICTRL_PROP Active

Short Description: Info for Field Control

Properties Definition Value Range

Single Vals

I	Fixed	Short Description
		Hidden (obsolete)
*		Output Only (obsolete)
.		Optional (obsolete)

< >

Dictionary: Display Domain

Domain: USMD_UICTRL_PROP Active

Short Description: Info for Field Control

Properties Definition Value Range

Single Vals

I	Fixed	Short Description
+		Required Entry Field (obsolete)
C		Optional
R		Output Only

< >

Dictionary: Display Domain

Domain: USMD_UICTRL_PROP Active

Short Description: Info for Field Control

Properties Definition Value Range

Single Vals

I	Fixed	Short Description
R		Output Only
M		Required Entry Field

< >

Flex Data Model

Active area is either blank or MDG.

No temporary key assignment for type 1.

Data Model Deletion:

You can only delete a data model if there is not an active version of the data model. Use the program **USMD_DELETE_DATA_MODEL** to first delete the active version of the data model and then delete the data model by selecting that data model in Edit Data Model and clicking on the Delete icon. This will delete the data model and the staging area but it will not delete the staging structures. You have to

manually delete the staging structures. Just go to SE11, enter the staging structure name and click on the delete icon.

Standard Data Models:

Out-of-the-box data models with standard SAP MDG License:

- 0F – Chart of Accounts (obsolete)
- 0G – MDG Finance
- MM – Material Master
- BP – Business Partner

Out-of-the-box data models with Utopia/Prometheus MDG License:

- U1 – EAM (Enterprise Asset Management)
- U2 – SM (Service Master)
- AR – Article Master

Activities to be done by MDG functional consultant to enable the data models in the system:

1. Activate the Business Function (transaction SFW5, done by BASIS team). This needs to be activated in each environment.
2. Once business function is activated, data model will get imported into the system with active version “NONE”.
3. We should manually activate the data model in each environment (cut over activity).

USMD_COMPARE_DATA_MODEL ? compare active and inactive versions of data model.

Process Modelling

Modelling the governance process for MDG Business scenarios.

In MDG, we don't create master data directly and get that available to the end users. The master data process should undergo certain levels of validations/approvals. In process modelling, we will be defining the entire governance process flow for the master data operations.

Process modelling is about setting up end-to-end master data maintenance process in SAP MDG from a central master data governance point of view. A key aspect of central master data governance is setting up end-to-end process to maintain, create, read, update, delete (CRUD) operations of master data objects and how workflows are configured to support the end-to-end processes.

Example:

Create material ? create CR 100 ? choose CR type ? configured for 3 steps approval process ? defined in workflow.

Agenda:

What is Change Request?

Types of Change Requests

Pre-requisites to use MDG Governance Process (workflows in MDG) – baseline configuration

Types of workflows in MDG

Static workflows vs rule-based workflows

Static workflow CR type configuration

Rule based workflow (RBWF) CR type configuration

- Simple RBWF)

- Dynamic agent selection

- Dynamic rule context

- Email notification ? using system caller BADI / using sub-workflows

- Parallel workflows

- Parallel CR

- Successor/follow-up CR

Workflow troubleshooting

Pre-requisites to use MDG Governance Process (workflows in MDG) – baseline configuration

1. Configure workflow tasks – done by MDG team

IMG Path: MDGIMG ? General settings ? process modelling ? Workflow ? configure workflow tasks.

Just focus on CA-MDG-AF (MDG Application Framework).

Click on Assign Agents.

You will see all the general task and background tasks of the application framework.

General task/dialog tasks are for the foreground users and background tasks are for the system users.

MDG user the workflow system batch user (WF-BATCH/SAP_WFRT) to process background tasks.

2. configure workflow background user (t-code SWU3) – done by basis team

System user **SAP_WFRT** takes care of the workflow related background tasks.

3. activate Event Type Linkages (t-code MDGIMG) – done by MDG team

IMG Path: MDGIMG ? General settings ? process modelling ? Workflow ? activate Event Type Linkages.

Configuration should be done only for Business Object **BUS2250** (Business Object for MDG Change Requests).

T-code SWO1 for Object Type display. Enter BUS2250 and display.

Type Linkage Active indicator should be active for the events CREATED, ACTIVATED & ROLLED BACK.

And Enable Event Queue should be active for the events ACTIVATED & ROLLED BACK.

Governance Scope

once you activate your data model, it will be present in this section. The column governed for each entity type checks whether that entity type is governed or not. If YES is chosen, then you can edit governed fields in change requests. You can never edit ungoverned fields. This field remains disabled i.e., the user can see the field but cannot change it.

In Customizing, you cannot access or remove the following fields from the governance scope:

- Key fields
- Required fields

You can also not remove entity types with a **storage and use type** of 1 from the governance scope. You can, however, remove their attributes and referencing relationships from the governance scope.

Any changes made to the governance scope will affect how the fields are handled during the change request process and during data loads. Following is a summary of the impact:

- **Single record processing** - attributes removed from governance automatically changes to read-only mode in the UI, and all related checks are disabled.
- **Mass Change** - ungoverned fields cannot be updated using the mass change process.
- **File upload** - all ungoverned fields are ignored.
- **File download** - ungoverned fields are included in the download file. However, the comments section the download file indicates that fields weren't part of the governance process.
- **Data export** - ungoverned fields are included in the exported file.
- **Data import** - if the data is imported directly into the active area, then governed as well as the ungoverned fields are updated. However, if the governance option is selected, and the imported data goes through a change request process, the ungoverned fields are ignored.

Change Request Header

This section has three default sections/tabs: General, Notes, and Attachments.

General ? basic details of the CR like CR number, status, description, priority, due date, reason, etc.

Notes ? any note written by the requestor, reviewer, approver.

Attachments ? attach files/links.

CR Processing

What is change request?

It is a container of changes made to a master data object that contains additional details on the relevant process. A CR supports day-to-day CRUD operations by associating a pre-defined process, data enrichment and data quality rules.

Types of CRs

Linear CR – you cannot have multiple CRs at a time for the same business object.

Parallel CR – having multiple CRs at a time for the same business object.

Requester (person who initiates the CRUD operation) ? once the requester submits, it goes to the next team called **Master Data experts** who **maintain, enrich & validate the data** ? once data is validated, then next team is **Master Data Steward** who **approves** the data ? once approved, data gets **replicated** to the target system.

Purpose of CR Type

To define a step-by-step governance process flow to carry out business operations on master data.

How to framework knows which process flow must be executed when we choose a CR type during master data operation?

We assign Workflow Template for each CR Type in the backend.

Workflow Template

SAP delivered two types of workflow template for MDG.

1. static workflow template

t-code for Workflow Builder **SWDD**.

Workflow template with pre-defined and consistent governance process flow.

2. rule-based workflow template

Workflow template without pre-defined and consistent governance process flow (purely dynamic). (Example: **WS60800086. This is the only rule-based workflow template**)

Configuring Custom CR Type with static workflow template

Before creating, know the client requirements. Example requirement for this case is: create new material with 3-steps governance process (requester → reviewer → approver).

Step 1: [MDGIMG](#) → [General Settings](#) → [Process Modelling](#) → [Change Requests](#) → [Change Request Types](#).

Step 2: Workflow Template that we will use for this is WS75700040.

Step 3: know the below details before configuring the CR types.

Data Model: MM

Type 1 entity: MATERIAL

Business Activity: Create material.

Processing Mode: single object processing

Step 4: you can either click on existing CR type and click on Copy button and change the name and other necessary things or click on New Entries. Enter name of your custom CR Type.

Step 5: Edition Type column is only for Finance object.

Step 6: Enter the Data Model and give a description for the CR type.

Step 7: if Single Object checkbox is selected then it means this CR can process only a single object at a time (single object processing).

Step 8: Enter the Main Entity Type and Workflow Template.

Step 9: Define the entity types for the CR Type. Enter MATERIAL as the entity type and give the standard configuration ID (**MDG_MM_APP_BS_MAT_GEN**).

Note that the field *Configuration ID* in the view *Entity Types* is only relevant if you use the obsolete Web Dynpro application UMSD_ENTITY_VALUE2.

Step 10: Message Output has two options: Standard (default) and Issue Error Messages as Warnings. If you select Standard, then during validation it will display errors as error messages. And if you want to bypass the errors then choose then 2nd option which will display the errors as warning messages.

Step 11: select MAT1 (standard create material) as the Business Activity.

If you save this, then you will get the below warning message.

No processor assigned to Workflow WS75700040 for change request type ZMAT1.

You can ignore this and save the changes.

Step 12: assign the processors to the CR Type.

[MDGIMG](#) → [General Settings](#) → [Process Modelling](#) → [Workflow](#) → other MDG workflows → [assign processor to Change Request step number \(simple workflow\)](#).

Step 13: click on New Entries. Enter the CR type. Step column has 4 options: 0 – submission, 1 – processing, 2 – final check, 3 – revision. For each step, you can choose the relevant object type (Job, organizational unit, position, user) and enter their Agent ID. For Requester step it is optional. For other steps, you can enter multiple users.

Now you can test by creating a material with this CR type by going to NWBC/Fiori → Material Governance.

Once you create and save the changes, the status will be “changes to be executed”. After you submit the workflow will trigger and you can see the status as “workflow is being processed”. Once the workflow is processed, the next status will be “changes to be executed” indicating now it is in Reviewer Step. You will see two buttons/actions below – Finalize Processing and Send for Revision. Click on Finalize

Processing. The next status will be Final Check to be Performed. You will see two buttons/actions below – Approve and Reject. Click on Approve. Once you approve, the CR will be finalized, and material number will be created.

Analyzing workflow template:

Go to tcode SWDD or SWDD_DISPLAY. Enter the workflow template ID (WS75700040). Press Enter.

Choose Graphical Model on your ride side. Double click on the first step (changes to be executed).

Double click on the task id under Control tab.

Under Basic Data: Each task is assigned with the Object Type BUS2250 and a method. When this task is executed at runtime, framework will execute the logic of this method.

Under Container: any pre-defined values will be shown here.

Every status in a workflow will have a different task ID and method assigned to it.

Business Activity and Logical Action

For all the business processes being displayed in the UI, there is an action code/event assigned in the UI level which decides what is the business process (create, delete, change, etc.).

Each CR type has an associated Business Activity.

Linkage between button (in UI) and CR type is Business Activity.

[General Settings](#) [Process Modelling](#) [Business Activities](#).

Business activity is created with a link to a logical action (CREATE, CHANGE, DISPLAY, LOCK, DELETE, CLEAN UP, DATA EXCHANGE, MASS PROCESSING), data model and business object type (147 – BP, 194 – MM), which means every business activity is unique to a data model and business object.

Logical Actions: A logical action represents the operation to be performed on the master data by an actor in the process (e.g., Create, Change, or delete) in the UI. Business activities add business context to logical actions by linking them with business objects such as create supplier, change material, and delete account.

Linkage between Logical Action, Business Activity and CR Type:

Example: We have a standard CR Type MAT01. This CR Type is associated with a Business Activity MAT1 (create material). This Business Activity is linked to a Logical Action (CREATE). These Logical Actions are pre-defined.

Rule-Based Workflow

Used for customized governance process flow.

Step 1: Requester ---submit/resubmit

Submit withdraw ---> stop and roll back changes

Step 2: Reviewer---approve?

If yes --->approver

if no goes back to requestor step

Step 3: Approver---approve?

If yes --->Activate

If no goes back to requestor step

Step 4: Activate---successful?

If yes--->Complete

If no ---> stop and roll back change

WS60800086 is the only rule-based workflow template.

Creating CR with RBWF

While creating custom CR with RBWF, you will get the below warning message:

No step defined for workflow WS60800086.

to define step, two ways:

i. [Processing Modelling](#) [workflows](#) [Rule based workflow](#) [Configure rule-based workflow](#).

ii. t-code USMD_SSW_RULE.

Both will take you to same web page.

Enter the CR type that you created with 86 wf template.

Click on Continue.

You will get the Business Rule Framework plus web page (BRFplus).

Whenever you configure a CR type with RBWF, a BRFplus application will be created. All the things will be defined by the framework itself.

Catalog that will be created is USMD_SSW_CATA_<your CR type>.

Our focus should be Decision Tables. Here we will define the entire flow.

Decision Tables:

i. **single value decision table** – Here we will define the flow.

System generates with the name DT_SINGLE_VAL_<CR Type>.

CR Previous Step	Previous Action	CR Priority	CR reason	CR Rejection Reason	CR Parent Step	Parallel Agt Grp No.
Condition Alias	New CR Status	Hours to Complete	Merge Type	Merge Parameter	Dyn Agt Sel Service	

If it is a dialog task, we need to assign user agents. For that, we will go to the next table under decision tables which is User Agent Decision Table and if it's a background task, we will go to Non-User Agent Decision Table.

ii. User Agent Decision Table

System generates with the name DT_USER_AGT_GRP_<CR Type>.

*Condition Alias	*User Agt Grp No.	*Step Type	*User Agent Type	*User Agent Value
------------------	-------------------	------------	------------------	-------------------

iii. Non-User Agent Decision Table.

System generates with the name DT_NON_USER_AGT_GRP_<CR Type>.

*Condition Alias	*Agent Group	*Process Pattern	Service Name
------------------	--------------	------------------	--------------

Table Settings [Condition Columns & Result Columns](#)

Condition Columns [currently what is happening.](#)

Result Columns [on this condition, what is happening.](#)

Condition Alias is used to link SDVT with UADT/NUADT.

The first step is always 00 (requester). If you don't assign any action to this step, the framework understands that 00 is the requester and the action performed in Submit.

For RBWF based CR type, we can give custom CR Step. Example, for Reviewer we can give RV.

These steps are defined under [General Settings](#) [Process Modelling](#) [Change Requests](#) [Configure Properties of Change Request Steps](#).

You can display the description for the CR step in Decision Table.

[General Settings](#) > [Process Modelling](#) > [Workflow](#) > [Define Change Request Step for Rule-based Workflow](#).

Click on New Entries. Enter your CR type. Enter CR step. Enter Description and Next Step.
Do the same for all the steps.

Always assign User Agent Grp in UADT as 001.

The Step Types in UADT is defined under [General Settings](#) > [Process Modelling](#) > [Workflows](#) > [Define Change Request Step Types and Actions](#).

What's Next button in CR > You can see the previous step, current step and next step.

Warning messages:

No golden path found.

This is the positive flow of the governance process for informative purpose.

No or invalid next steps defined.

Define the next steps for every step.

[General Settings](#) > [Process Modelling](#) > [Workflow](#) > [Define Change Request Step for Rule-based Workflow](#).

Parallel Workflows

In parallel workflows, a workflow is sent to multiple departments at the same time, each department takes an action, and when the action is finished, the result is merged to determine the next flow.

BAdI used is: **USMD_SSW_PARA_RESULT_HANDLER**

The Business Add-In (BAdI) USMD_SSW_PARA_RESULT_HANDLER is used to merge the result of the parallel workflow items. You can use this BAdI to implement the result of a parallel workflow merge in the rule-based workflow. The BAdI uses the method HANDLE_PARALLEL_RESULT to handle and merge the results of the parallel workflows into one result. The input for this BAdI is the change request number, current step number, parallel step action table, and the service name. By using the change request number, it is possible to access all data within this change request.

Usually, when approver1 rejects the CR, it still goes to other approvers which is not needed. We need to modify this workflow to handle instant rejection.

Get all the workitems which are pending in the current work item ID (at approval level) and close them and then the merge BAdI gets triggered. Every work item will have a parent work item known as top work item. To get the top work item of a CR, execute class

CL_USMD_WF_CREQUEST_MAPPER=>GET_TOP_WI_BY_CREQUEST_SINGLE.

To get sub work items for the top WI, execute the function module SAP_WAPI_GET_DEPENDENT_WIS.
Get the workitems which are in Dialog step (W) and status in 'ready'.

To close these workitems, use the function module **SAP_WAPI_WORKITEM_COMPLETE.**

For this, use synchronous system method BAdI.

Parallel Change Requests

- enables you to activate or reject a change request independently from the processing results of other change requests for the same business object.

- as of MDG 7.0 and SAP S/4HANA 1511, MDG also supports the creation of more than one CR in parallel for a single BO.

Advantage/use:

Suppose there are multiple teams for material processing like MDM team (responsible for basic data, dimensions), Planning team (plant mgmt.), Quality team (quality view, classifications), Sales team (sales views).

Suppose there is a material 123 initiating Change Process MAT02 CR# created 300.

If MDM team is editing the basic data, then other teams can't edit the basic data but they can edit other entity data like plant, sales etc. Hence, locking is enabled at entity level whereas, in case of linear CR process, locking is enabled at BO level (all entities are locked in one CR).

Limitations:

Not recommended for create process. Only applicable for change process.

Not applicable for finance object.

Configuration:

Can be configured using either static WF template or RBWF template.

Steps to create parallel CR:

Step 1: create a CR Type.

Step 2: to make this CR type a parallel CR, select the Parallel checkbox.

Error while saving:

CR Type ZPMAT02; Enter at least one entity type which is in scope.

Since you defined this CR type as parallel, you need to define the scope for this. Meaning, what data do you want to change in this CR.

Step 3: to define scope, select the entity type and click on Scope on Entity Type Level. For MATERIAL entity, MATERIAL only is the basic data entity so enter MATERIAL.

Error while saving:

Entity MATERIAL in scope, therefore entity UNITOFMSR must also be in scope.

Entity MATERIAL in scope, therefore entity MEAN_GTIN must also be in scope.

This is because, entities UNITOFMSR and MEAN_GTIN are dependent on MATERIAL entity.

Step 4: add these two entities also in Scope. Only these entities you can edit in CR (these are the entities that will be locked under this CR type. For other entities, you can create a different parallel CR).

Step 5: assign the processors.

Locking logic for parallel CR:

Non-parallel CRs, such as those created by "Create Master Data CR", completely lock the chosen business object. Likewise, to create a non-parallel CR, the entire master data record must be unlocked.

For example:

For materials in parallel CRs, locking happens on entity level, that is, on the object level of the entity type. In the background an object list is maintained for this. For example, if plant ATP data is maintained for plant 001 in the first parallel CR, then in second parallel CR, the plant ATP data is locked for plant

001, but not for plant 002. Examples of objects on entity level are plants, storage locations, sales org., distribution channels, warehouses, and storage types.

E-mail Notification

No standard configuration for e-mail notifications. Have to go and see manually.

NWBC ? Change Requests ? Change Requests and Documents ? Workflow Inbox.

This the place where the user can see all the CRs waiting for your action or already taken action.

Another place: NWBC ? Change Requests ? Change Requests and Documents ? My Change Requests.

Here, if any CR is wrongly assigned under your name, then you can “forward” it to other user.

You can “create substitution rule” for assigning the tasks that comes to your bucket to other user in case you go on planned leave.

Questions before implementing email notification:

What is the business object?

What is the business activity?

What is the governance process flow?

When to trigger the email notification?

Whom to send the notification?

What is the template of the email?

Process flow: Requestor ? Reviewer ? Approver.

Requirement: While creating new COMPANY (model OG; Entity – COMPANY), when approver rejects CR, an email notification should be sent to Requestor.

High level steps to implement Email Notifications:

1. Create CR type with assignment of WS*86 template.
2. Build BRF+ application
3. update the respective tables
4. create EMAIL NOTIFICATION service
5. Implement the system caller BAdI
6. test the requirement.

API (Application Programming Interface) Programming

What is an API?

Application Programming Interface or API is an interface provided by an application for interacting with other applications.

Frequent Scenarios of API Calls

1. Create Change Request
2. Read Change Request data or properties
3. Modify Change Request data or properties

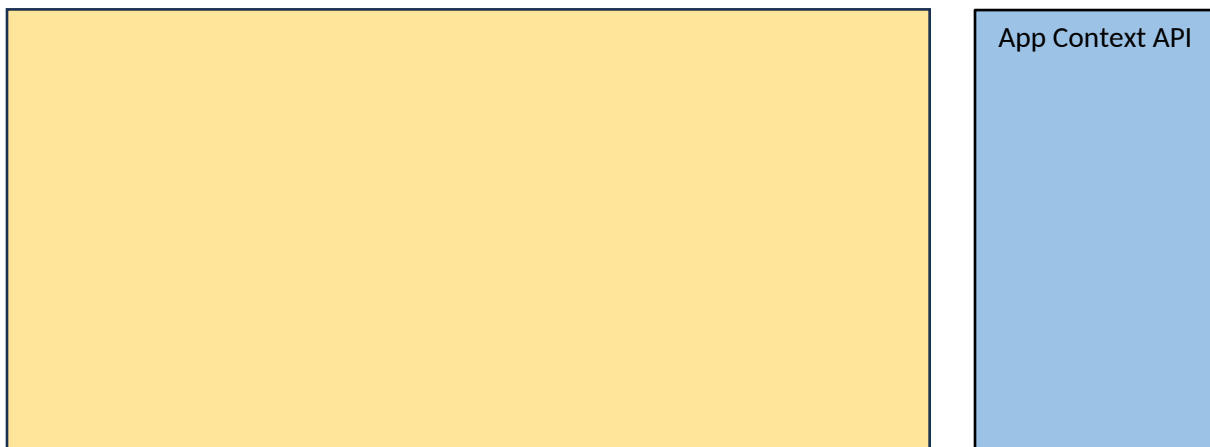
4. Read entity/entities data
5. Write the data to entities
6. Delete the data from entities
7. Lock the entity
8. Unlock the entity
9. Lock/unlock change request
10. Authorization checks, etc.

One of the frequent uses of API calls is create Change Request (supplier registration is initiated elsewhere and in comes to MDG system through some interface like BAPI, SOA, IDoc etc. and a CR is created in MDG).

In MDG, SAP has delivered APIs in the form of classes and interfaces.

Various APIs in MDG:

TITLE	NAME	PURPOSE
Change Request API	CL_USMD_CREQUEST_API IF_USMD_CREQUEST_API	To create CR, read/write CR header properties and to read CR data.
External Model API	CL_USMD_MODEL_EXT IF_USMD_MODEL_EXT	To read staging/entity data.
Governance API	CL_USMD_GOV_API IF_USMD_GOV_API	To handle governance scenarios of several change requests.
Convenience API	CL_USMD_CONV_SOM_GOV_API IF_USMD_CONV_SOM_GOV_API (SOM stands for Single Object Maintenance)	Same as governance API but using a single change request.
App Context API	CL_USMD_APP_CONTEXT IF_USMD_APP_CONTEXT	To store and provide the current context parameters of an applications such as CR ID, CR type, CR step, etc.



External Model API

Change Request API:

Method name	Purpose
GET_INSTANCE	Creates and returns instance of the class
IF_USMD_CREQUEST_API~ACTIVATE_CREQUEST	Activates the CR. "All or nothing principle"
IF_USMD_CREQUEST_API~ADD_ATTACHMENT	Adds attachments to the CR
IF_USMD_CREQUEST_API~ADD_NOTE	Adds notes to the CR
IF_USMD_CREQUEST_API~CHECK_CREQUEST	Checks CR
IF_USMD_CREQUEST_API~CREATE_CREQUEST	Generates CR
IF_USMD_CREQUEST_API~GET_CREQUESTS	Gets the list of CRs based of selection criteria
IF_USMD_CREQUEST_API~GET_CREQUESTS_FOR_OTC	Gets the CRs for OTC code
IF_USMD_CREQUEST_API~READ_CREQUEST	Reads CR
IF_USMD_CREQUEST_API~READ_VALUE	Reads the master data
IF_USMD_CREQUEST_API~RETRIEVE_CREQUEST	Retrieves list of CRs of one object instance
IF_USMD_CREQUEST_API~SAVE_CREQUEST	Saves CR
IF_USMD_CREQUEST_API~SET_ATTRIBUTES	Sets the priority of the CR
IF_USMD_CREQUEST_API~WRITE_VALUE	Writes the master data.

Difference between Governance API and Convenience API

Convenience API	Governance API
Only one CR processing is allowed at a time.	Multiple CR processing is allowed at a time.
We can process only single object at a time.	We can process multiple objects at a time.
When we are using instance of convenience API, should not use governance API.	When we are using instance of governance API, should not use convenience API.

UI Modelling

Evaluation of User Interfaces in SAP – SAP GUI

- In 1970s, SAP came into market as an ERP product with its own user interface SAP GUI.
- In 1970s, there was no Browser Revolution. Browser Revolution started in 1990s. SAP had the following big challenge to move from GUI to Browser based technologies.
- Interaction between SAP GUI and Application server is **through DIAG (Dynamic Information and Action Gateway) protocol**. But browser don't understand DIAG protocol. It can understand only HTTP protocol.
- Hence SAP started with **ITS (Internet Transaction Services) Applications**, which enable the users to access SAP Transaction through Web.
- ITS Applications converts HTTP protocol to DIAG protocol and vice versa.
- The drawback with ITS is that externally we need to have an additional server (web server).
- To overcome this drawback, SAP replaced ITS **with ICM (Internet Communication Manager) which contains ICF (internet communication framework)** to convert HTTP protocol to DIAG protocol and vice versa. BSP/webdynpro/Fiori applications use ICF.
- Technically, SICF transaction leverages to activate/deactivate all the services related to BSP/webdynpro applications.

Introduction to Webdynpro ABAP

Webdynpro – web-based screen programming. Can be built using either JAVA or ABAP.

Webdynpro ABAP is SAP Standard UI Technology used to develop Web Applications in ABAP environment.

Advantages:

- It reduces the implementation effort.
- Clear separation of layout and business data.
- Reusability of the components.
- Automatic data transfer using Context Binding.
- It supports stateful applications i.e., even if the page changes, the data in the previous page still remains.

Architecture used by webdynpro applications is **MVC (Model View Controller) Architecture**. Fiori also uses this architecture.

MVC Architecture

MVC Design pattern contains a clear distinction between Processing Control, Data Model and displaying the data in UI.

View – User Interaction Layer – Generates the application data without caring how it will be generated.

Controller – Binding Layer – Binds the user and business interaction layers together. All intermediate processing is performed here.

Model – Business Interaction Layer – Generates the application data without caring how it will be displayed.

Building Webdynpro Applications

Step 1: Tcode – SE80.

Step 2: Select Web Dynpro Components/Interfaces from the drop-down menu.

Step 3: Enter the name of your application. Press Enter and create the component.

Step 4: Now activate the created component.

Assistance Class – place where you can write the business logic.

Types of controllers:

- ❑ Component controller (global controller)
- ❑ Window controller (local controller)
- ❑ View controller (local controller)
- ❑ Interface controller (cross-component controller)
- ❑ custom controller (global controller)

Designing Layout

Layout can be designed at view level.

Double click on the View. Go to Layout tab. You will find various elements there like text, captions, label etc.

Mandatory properties for any UI element (button, text, icon, link, etc) are highlighted in red color.

Parent element is **ROOTUIELEMENTCONTAINER**.

Property: Layout – Value: FlowLayout ☐ it means all the elements will be displayed in a flow one after the other (in a single line without any gaps). If you want to display line by line then use MatrixLayout.

For BUTTON element type, events must be created. Events/actions are nothing but the method.

To generate URL, right click on the component name -> create -> Web Dynpro Application. Enter application name and description and continue. Save. Application is created. Double click on the component name, go to Properties tab. You will find the URL under Administrative Data.

Once URL is generated, service gets created with the application name and gets added in SICF.

Introduction to FPM

- ☐ FPM is based on the web Dynpro technology paradigm, and it helps developers build UIs on the ABAP application layer with a standardized layout/look and feel.
- ☐ FPM is a UI framework, that is, a set of tools, templates and classes, which ensures a more consistent look and feel to the user interfaces of SAP applications. Applications built using FPM conform to the latest SAP UI and accessibility guidelines.
- ☐ It also provides more flexibility in various configuration options.
- ☐ FPM framework provides the ability to create or adapt the UI configurations in less time.
- ☐ The key components include SAP pre-defined floorplans, UI building blocks, and the Floorplan Manager Configuration Editor (flexible user interface designer FLUID).
- ☐ A floorplan can consist of numerous UIBBs. The UIBBs provide a standardized look and feel for form layout, list layout, search etc.
- ☐ A section consists of a number of UIBBs.

Types of Floor Plans:

- ☐ Overview Page Floor Plan (OVP) – one after the other. Most widely used.
- ☐ Guided Activity Floor Plan (GAF) – roadmap kind of page.
- ☐ Object Instance Floor Plan (OIF)/ Quick Activity Floor Plan (QAF) – mostly consists of tabs.

Associated Webdynpro components:

- ☐ OVP ☐ FPM_OVP_COMPONENT
- ☐ OIF ☐ FPM_OIF_COMPONENT
- ☐ GAF ☐ FPM_GAF_COMPONENT

Classification of UIBBs:

- ☐ **Generic UIBBs** – include templates for forms, lists, etc.

- **Reuse UIBBs** – include business logic along with UIs. Some examples include notes, attachments, etc.
- **Freestyle UIBBs** – usually generic web Dynpro component assigned to the FPM application using the IF_FPM_UI_BUILDING_BLOCK web Dynpro interface.

Creating simple FPM Application (non-MDG specific)

T-code is **FPM_WB** (FPM Workbench). Here we can create a new FPM application or edit an existing one.

Alternate way to launch FPM Workbench is execute the t-code SICF → Enter the service FPM_WB → Apply → Test the service.

Click on **Wizard for Creating Empty FPM Applications**. Enter name and description of your webdynpro application. Then choose the floor plan and enter configuration name for Application and Floorplan Configuration. Save it.

Go to SE80 → choose webdynpro components/interface → enter the default component name (OVP/OIF/GAF) and Enter. Your created application will be under Component Configurations.

Feeder Class → provides a link between the application and the generic UIBB. It contains the business logic behind an application. Main purpose is to feed the presentation logic to the UIBB.

Interface for the feeder class → IF_FPM_GUIBB

For form → IF_FPM_GUIBB_FORM

Go to FPM Workbench and click on **Configure Application**. Enter your application name and configuration ID and Enter.

An application has application configuration associated with it. This application configuration has floor plan configuration associated with it. This floor plan configuration has FPM pages and each page has the relevant UIBB configuration.

Standard UI Application

The links present in NWBC are called as roles. Roles are assigned to user ID.

Business Rules

Types of business rules in MDG:

1. **Validation Rules** – validating the data – code based (BADI implementation, BRF+).
 - a. Single/Same Entity Validation Rules (BADI used is **USMD_RULE_SERVICE**)
 - b. Cross Entity Validation Rules (BADI used is **USMD_RULE_SERVICE_CROSS_ET**)
2. **Derivation Rules** – deriving data based on data entered by user – code based (BADI implementation, BRF+).
 - a. Single/Same Entity Derivation Rules (BADI used is **USMD_RULE_SERVICE**)
 - b. Cross Entity Derivation Rules (BADI used is **USMD_RULE_SERVICE_CROSS_ET**)
3. **UI handling Rules** – handling/controlling the UI handling properties – code based (BADI implementation, configuration).

- a. Single/Same Entity UI handling Rules (BAI used is **USMD_ACC_FLD_PROP_CUST_DEP_SET**)
 - b. Cross Entity UI handling Rules (BAI used is USMD_ACC_FLD_PROP_CUST_DEP_SET)
 - c. Generic Entity UI handling Rules (BAI used is USMD_ACC_FLD_PROP_CUST_DEP_SET)
4. **Key Mapping Rules** – mapping key fields of type 1 entity. Applicable only for hub implementation – configuration based (BAI implementation, mostly configuration).
5. **Value Mapping Rules** – only for non-key fields (plants, company codes, etc.). Applicable only for hub implementation – configuration based (BAI implementation, mostly configuration).
6. **Replication Rules** – Applicable only for hub implementation – configuration based (BAI implementation, mostly configuration).

For validation and derivation rules, MDGIMG ? General Settings ? Data Quality and Search ? BAI : Define Validations/Derivations.

For UI handling rules, MDGIMG ? General Settings ? Process Modelling ? Change Requests ? Business Add-Ins ?BAI : Access to Customer-dependent field property settings.

Requirement: for a specific CR type, at requestor level (00 step number), client wants to make notes/attachments mandatory at CR header level.

BAI used for this purpose is USMD_RULE_SERVICE (this is the definition).

MDGIMG ? General Settings ? Data Quality and Search ? BAI : Define Validations/Derivations ? create new. Enter the enhancement implementation name starting with Z.

Next, enter BAI implementation name and implementation class. Continue.

Now, BAI implementation screen opens. Double click on Filter Values ? Combination. Since the validation is required at CR header level, we can choose filter as entity type = 1. The other one is Model. Save it. Next double click on the Implementation Class and provide empty implementation as those are the interfaces (double click on each method and save it).

If you want to implement single entity validation or cross entity validation, then CHECK_ENTITY method will be implemented. Double click on it.

Parameters in CHECK_ENTITY:

IO_MODEL ? TYPE REF TO IF_USMD_MODEL_EXT ? Model extension API used to read the staging data.

ID_EDITION ? TYPE USMD_EDITION ? specific to finance only.

ID_CREQUEST ? TYPE USMD_CREQUEST OPTIONAL ? Current change request.

ID_ENTITYTYPE ? TYPE USMD_ENTITY ? current entity type.

IF_ONLINE_CHECK

IT_DATA ? TYPE ANY TABLE ? Current entity's data will be stored.

ET_MESSAGE ? TYPE USMD_T_MESSAGE.

Double click on this CHECK_ENTITY method and write the logic for the above requirement.

This class does not have change request type. There are different ways to get the CR type. One of the ways is by using context API (class CL_USMD_APP_CONTEXT).

1st step is getting the CR type and step number.

DATA: lo_context TYPE REF TO if_usmd_app_context,
lo_crequest_api TYPE REF TO if_usmd_crequest_api.

```
DATA: lt_notes TYPE usmd_t_crequest_note,  
      lt_attachments TYPE usmd_t_crequest_attachment.
```

```
DATA: ls_message TYPE usmd_s_message.
```

```
CALL METHOD cl_usmd_app_context=>get_context  
RECEIVING  
eo_context = lo_context.
```

```
IF lo_context->mv_crequest_type EQ '<CR TYPE>' AND lo_context->mv_crequest_step EQ '00'.  
ENDIF.
```

Next, you need to read the CR header data (notes and attachments mainly). For this you can use CR API which is CL_USMD_CREQUEST_API method READ_CREQUEST. This method is instance method so you need to call the GET_INSTANCE method first.

```
IF lo_context IS BOUND AND lo_context->mv_crequest_type EQ '<CR TYPE>' AND lo_context->  
>mv_crequest_step EQ '00'.  
**first instantiate the change request API  
    CALL METHOD cl_usmd_crequest_api=>get_instance  
        EXPORTING  
            lv_crequest      = id_crequest  
            lv_model_name    = 'MM'  
        IMPORTING  
            Re_inst_crequest_api = lo_crequest_api.  
    IF lo_crequest_api IS BOUND. "if creation of object is successful, then read CR header data.  
        CALL METHOD lo_crequest_api->read_crequest  
            IMPORTING  
                et_notes      = lt_notes  
                et_attachment = lt_attachments  
        **validate notes and attachments. If either not available, throw error  
        IF lt_notes IS INITIAL AND lt_attachments IS INITIAL.  
            **initiate msgid, msgno, msgty  
            ls_message-msgid = '<msg-id>'.  
            ls_message-msgty = 'E'.  
            APPEND ls_message TO et_message.  
            CLEAR ls_message.  
        ENDIF.  
    ENDIF.  
ENDIF.  
ENDMETHOD.
```

To debug this, put external breakpoint on your ID and execute the CR.

Done with class 74.

SMT (Service Mapping Tool)

SMT is a tool used to map data between two structures with different field names.

E.g., we have two structures staging structure and active area. In staging structure there is a field ZMARA-MATERIAL. When the CR is finalized, we want this field to be updated in active area MARA-MATNR.

Data Replication Framework (DRF)

What kind of replications are available in MDG?

1. **Self-Replication:** Mandatory for Flex Solution in Co-deployment environment.
Use case scenario: We have dev system MD1-100 client. Cost center is being processed. Master data table in CEPC. CR 123 is the CR for this cost center. This CR is processed completely but it stays in the staging table only not in the active area (CEPC). So self-replication is configured here, i.e., replicating the data from staging table to active area within the same system (co-deployment).
2. **Target Replication/Hub replication:** Applicable for the hub environment.
Use case scenario: MD2-100 is MDG hub system connected to multiple other systems. The governed data (flex/re-use) must be replicated to the connected operational systems. In hub environment, the self-replication for flex data is optional.

DRFIMG or MDGIMG ? General Settings ? Data replication.

DATA MODEL ENTITIES

In SAP Master Data Governance (MDG), entity types represent the core building blocks of the data model. They define the master data objects that can be managed within the MDG system. There are four main types of entity types in SAP MDG, each with distinct characteristics:

1. **Type-1 Entity Type:** Represents the root objects that are subject to governance in MDG. Can be created, changed, and deleted through change requests. Examples of type-1 entity types include customers, materials, and vendors. Storage and Use Type: 1 Changeable: Via change request Generated Tables: Database tables
2. **Type-2 Entity Type:** Used to qualify type-1 and type-4 entity types by adding additional key attributes. Cannot be maintained directly in MDG, but their attributes can be used to filter and search for type-1 and type-4 entities. Examples of type-2 entity types include plant and sales organization for a material. Storage and Use Type: 2 Changeable: Not directly in MDG Generated Tables: Check/Text tables
3. **Type-3 Entity Type:** Similar to type-2 entity types, used to qualify type-1 and type-4 entity types. However, type-3 entity types can be maintained indirectly in MDG through their relationships with type-1 entities. An example of a type-3 entity type is a country code for a customer Storage and Use Type: 3 Changeable: Indirectly through type-1 entities Generated Tables: No generated tables

4. Type-4 Entity Type: Represents dependent objects that are associated with type-1 entities. Cannot be processed directly in MDG, but their data can be maintained indirectly through the related type-1 entity. Examples of type-4 entity types include bill-of-materials (BOM) components for a material and contact persons for a customer. Storage and Use Type: 4 Changeable: Via other entity type Generated Tables: Database tables.